

Introduction

Web applications continue to be one of the most common targets for cyberattacks. Because of this, understanding and identifying common vulnerabilities is critical for both security and compliance teams.

In this lab, we explored the OWASP Top 10, which highlights the most critical web application security risks. The goal was not just to find vulnerabilities but to understand their real-world impact from a governance, risk, and compliance (GRC) perspective.

Scenario

We were tasked with assessing a legacy web application server hosted on the OWASP Broken Web Applications platform.

With an upcoming organizational merger, this system needs to meet updated security and compliance standards. My job was to identify key risks, map them to the OWASP Top 10 and explain their potential impact on the business.

Objectives

The main goals of this assessment were to:

- Identify vulnerable web applications on the target system
- Configure and use Burp Suite Professional to analyze web traffic
- Detect vulnerabilities aligned with the OWASP Top 10
- Understand the business and compliance implications of each issue
- Present findings in a clear and structured format

Lab Setup

The lab environment consisted of:

- **Attacker Machine:** Kali Linux
- **Target Machine:** OWASP Broken Web Applications
- **Burpsuite:** Burpsuite Professional

This setup allowed us to simulate a real-world penetration testing scenario in a controlled environment.

Phase 1: Discovery and Reconnaissance

Step 1: Target Discovery

The first step was to confirm that the target system was active and reachable. This was done using a simple ping command to verify connectivity.

From a GRC perspective, this step is essential because you can't protect what you don't know exists.

Step 2: Service Enumeration

Next, we scanned the target using Nmap to identify open ports and running services.

This helped us understand:

- What services were exposed
- Where potential entry points existed

We also saved the scan results for documentation and reporting purposes.

Step 3: Application Discovery

After identifying open web services, we accessed the target through a browser.

This revealed multiple vulnerable applications, including:

- BodgeIt Store
- WebGoat

These applications served as the focus for our testing.

Phase 2: Tool Configuration and Scanning

Step 4: Configuring Burp Suite

We configured Burp Suite Professional to intercept traffic between the browser and the target application.

By routing traffic through Burp, we were able to:

- Capture requests
- Analyze responses
- Modify data in real time

Step 5: Automated Scanning

We then used Burp's scanning feature to perform an initial automated assessment.

This helped identify:

- Common vulnerabilities
- Misconfigurations
- Low-hanging security issues

The scan results provided a useful starting point for deeper manual testing.

Phase 3: Manual Testing (OWASP Top 10)

Step 6: Broken Access Control (A01)

We tested whether restricted areas could be accessed without proper authorization.

An administrative page was accessible without authentication, showing a clear failure in access control.

This is a serious issue because it allows unauthorized users to gain elevated privileges.

Step 7: Cryptographic Failures (A02)

We analyzed traffic in Burp and found that sensitive data, including passwords, was being transmitted in plain text. This means attackers could easily intercept user credentials.

Step 8: Injection (A03)

We tested input fields for SQL Injection vulnerabilities.

By modifying input values, we were able to manipulate database queries and retrieve unintended results. This is one of the most dangerous vulnerabilities, as it can lead to full database compromise.

Step 9: Authentication Failures (A07)

We tested the login system using weak credentials and basic brute-force techniques.

The application allowed weak passwords, making it easier for attackers to gain unauthorized access.

8. Step 10: Triage and Risk Assessment

After identifying the vulnerabilities, we evaluated their severity and potential impact using the **OWASP Top 10 framework**.

OWASP Category	Vulnerability	Application	Risk Level	Compliance Impact	Business Impact
A01: Broken Access Control	Unprotected Admin Page (/admin.jsp)	BodgeIt Store	High	PCI DSS 7.2.1	Unauthorized access to sensitive functions and data
A02: Cryptographic Failures	Passwords sent in plain text	WebGoat	High	PCI DSS 4.1	Risk of credential theft and account compromise

A03: Injection	SQL Injection via search input	BodgeIt Store	High	PCI DSS 6.5.1	Potential full database compromise
A07: Identification & Authentication Failures	Weak passwords / brute-force risk	BodgeIt Store	Medium-High	NIST CSF PR.AC-1	Increased likelihood of unauthorized access

Key Observations:

- Critical vulnerabilities such as SQL Injection and Broken Access Control pose immediate risks
- Lack of encryption exposes sensitive user data
- Weak authentication increases the likelihood of compromise

From a business standpoint, these issues could result in:

- Data breaches
- Regulatory penalties
- Loss of customer trust

Overall, the system was assessed as high risk and requires urgent remediation.

Executive Summary

To: CISO & Application Development Management

From: GRC Audit Team

Subject: Security Assessment of Legacy Web Applications

Overview

A security assessment was conducted on a legacy web application environment hosted on the OWASP Broken Web Applications platform.

The goal was to evaluate its security posture and identify risks based on the OWASP Top 10.

Key Findings

The assessment revealed several critical vulnerabilities:

- **SQL Injection (A03):** Could allow attackers to access or manipulate the entire database
- **Broken Access Control (A01):** Administrative functions were accessible without proper authorization
- **Cryptographic Failures (A02):** Sensitive data was transmitted without encryption
- **Authentication Weaknesses (A07):** Weak password controls increased the risk of unauthorized access

Business Impact

These vulnerabilities expose the organization to serious risks, including:

- Data breaches
- Regulatory non-compliance (PCI DSS, NIST)
- Financial loss
- Reputational damage

Recommendations

To address these issues, the following actions are recommended:

- Fix identified vulnerabilities using secure coding practices
- Enforce strong authentication and access controls
- Implement encryption for all sensitive data
- Integrate security testing into the development lifecycle
- Provide security training for developers

Conclusion

The current state of the application environment presents a **high security risk**. Immediate remediation and long-term improvements are necessary to ensure compliance and protect organizational assets.